BUDGET TRAVEL SCOUT

A Multi-Agent AI System for Budget-Friendly Travel Planning

Group 6

Simran Abhay Sinha
Arav Pandey
Savan Sangamesh Awanti

sinha.sim@northeastern.edu
pandey.ara@northeastern.edu
awanti.s@northeastern.edu

Submission Date: 17 April 2026

GitHub

Spring 2026

Table of Contents

1. Introduction

Budget Travel Scout is a multi-agent AI application designed to help travelers evaluate budget-friendly weekend trips. Given a starting city, destination, travel interest, optional daily budget cap, and trip duration, the system researches flight and hotel costs, discovers free or cheap local activities, and synthesizes everything into a comprehensive daily budget breakdown with a budget-friendliness score.

The project demonstrates core concepts of agentic AI: multiple specialized LLM-powered agents collaborating in a sequential dependency chain, augmented by a Retrieval-Augmented Generation (RAG) pipeline that grounds LLM responses in real web-scraped data rather than relying solely on model knowledge.

1.1 Problem Statement

Planning a budget trip is an overwhelming and fragmented experience. Travelers must separately search for flights, hotels, and activities across multiple websites, with no single tool providing a unified daily cost breakdown or budget-friendliness score. Generic travel recommendations fail to match specific interests, and there is no fast way to compare destinations by affordability. The average traveler spends over 3 hours planning a weekend trip budget. Budget Travel Scout completes the same task in under 10 seconds.

1.2 Project Objectives

- Build a 3-agent AI pipeline where Agent C depends on the outputs of both Agent A and Agent B, enforcing a real sequential dependency chain
- Implement RAG with live web scraping to ground LLM estimates in real destination-specific data
- Support budget constraint mode with over/under indicators and savings tips
- Deploy a permanent, production-accessible UI via Hugging Face Spaces
- Produce a polished HTML report with budget score, cost breakdown, and activity recommendations

2. Technology Stack

The system is built entirely in Python, leveraging production-grade tools across AI, data retrieval, and deployment layers.

Layer	Technology	Purpose
LLM	OpenAI GPT-4o	Powers all 3 agents with differentiated system prompts, temperatures (0.2/0.3), and task scopes
Vector DB	ChromaDB (in-memory)	Stores embedded web content with cosine similarity search for RAG retrieval
Embeddings	all-MiniLM-L6-v2 (SentenceTransformers)	Generates 384-dim semantic vectors from scraped text chunks for ChromaDB indexing
Web Scraping	BeautifulSoup4	Scrapes Wikivoyage wiki pages and Nomadic Matt travel blog for destination-specific content
UI Framework	Gradio	Interactive web interface with progress bars, examples, and HTML report rendering
Deployment	Hugging Face Spaces	Permanent public URL with API key secured via HF Secrets; eliminates Gradio 72-hour expiry
Notebook	Google Colab	Fully documented cell-by-cell walkthrough with markdown explanations for development

3. System Architecture

The application follows a sequential multi-agent pipeline where each agent performs a specialized task. The overall data flow is:

User Input → RAG Pipeline (Scrape → Chunk → Embed → Index) → Agent A (Researcher) → Agent B (Local Guide) → Agent C (Budgeter) → HTML Report

Agent C has a hard dependency on both Agent A and Agent B: it cannot execute until both upstream agents have completed and returned their structured JSON outputs. This enforces a real sequential dependency chain, not just a conceptual one.

3.1 RAG Pipeline

When a user enters a destination, the system executes a five-stage RAG pipeline before any agent runs:

1. Scrape	BeautifulSoup fetches the destination's Wikivoyage page and Nomadic Matt travel blog posts. Sections are extracted by heading (Overview, Budget, Eat, Sleep, etc.) to preserve topical structure.
2. Chunk	Raw text is split into 500-character overlapping chunks. Overlapping ensures that sentence-boundary context is not lost between adjacent chunks. Each chunk is deduplicated by MD5 hash to prevent redundant embeddings.
3. Embed	The all-MiniLM-L6-v2 model from SentenceTransformers generates 384-dimensional semantic vectors for each chunk. This model runs locally in Colab at zero cost.
4. Index	Chunks and their embeddings are upserted into a ChromaDB in-memory collection using cosine similarity as the distance metric.
5. Retrieve	When an agent needs context, rag_query() performs a cosine-similarity search and returns the top-k most relevant chunks (clamped to actual collection size to avoid errors). These chunks are injected into the agent's prompt as grounding context.

If scraping returns zero results (e.g., the destination has no Wikivoyage page), the pipeline seeds ChromaDB with 5 fallback chunks containing generic budget travel information so the vector store is never empty.

3.2 Agent Design

Agent A — Researcher (GPT-4o + RAG)

Agent A estimates round-trip economy flight costs and nightly hotel rates for the given origin-destination pair. It receives RAG context containing scraped price data and accommodation

information. When a max daily budget is set, its system prompt instructs it to flag if the destination is likely over budget. Temperature is set to 0.2 for precise, consistent cost estimates.

Output schema: {"flight_low", "flight_high", "hotel_low", "hotel_high", "booking_tip", "budget_warning"}

Agent B — Local Guide (GPT-4o + RAG)

Agent B recommends exactly 3 free or cheap local activities matching the traveler's selected interest (e.g., History, Nightlife, Food & Drink). It receives RAG context containing scraped activity and attraction data. On tight budgets (< $80/day), its prompt mandates all activities be completely free. Temperature is 0.3 for slightly more creative but still grounded suggestions.

Output schema: [{"name", "description", "cost", "source"}]

Agent C — Budgeter (GPT-4o)

Agent C is the synthesis agent. It receives the complete outputs of Agent A (cost data) and Agent B (activity list) and computes a daily budget breakdown including hotel, food, transport, activities, and flight amortization. It calculates a budget-friendliness score from 1-10, a total trip cost estimate, and an over/under budget indicator. When the daily total exceeds the user's max budget, it sets over_budget=true and generates an array of specific cost-cutting savings tips.

Output schema: {"daily_hotel", "daily_food", "daily_transport", "daily_activities", "daily_total", "flight_amortized", "grand_daily_total", "score", "summary", "over_budget", "savings_tips", "total_trip_cost"}

4. Code Flow

The application is organized into 6 sequential Colab cells. Each cell is self-contained and builds on the previous one.

Cell	Name	Description
Cell 1	**Install**	Installs openai, gradio, requests, beautifulsoup4, chromadb, and sentence-transformers via pip.
Cell 2	**API Keys**	Loads the OpenAI API key from Colab Secrets (userdata.get) and initializes the OpenAI client. No keys are hardcoded.
Cell 3	**RAG Pipeline**	Defines scrape_wikivoyage(), scrape_budget_tips(), chunk_text(), build_rag_knowledge(), and rag_query(). On each run, it scrapes 2 sources, chunks into 500-char segments, embeds with MiniLM, and upserts into ChromaDB. The rag_query function clamps n_results to collection.count() to prevent crashes.
Cell 4	**Agent Logic**	Defines agent_a(), agent_b(), agent_c() with parse_json() helper. Each agent calls OpenAI GPT-4o with a unique system prompt. RAG context is injected into Agent A and B prompts. Budget constraints are threaded through all three agents. Every agent has try/except with structured fallback data.
Cell 5	**Report Builder**	build_report() generates a styled HTML card with: gradient header showing route and trip params, Agent A cost ranges, Agent B activity table, Agent C budget metrics with score bar, over/under budget banner with color coding, savings tips list, RAG badge showing indexed chunk count.
Cell 6	**Gradio UI**	Defines run_pipeline() orchestrator and launches the Gradio Blocks interface. The pipeline runs: RAG build → Agent A → Agent B → Agent C → Report, with per-agent progress updates. Each agent is isolated in its own try/except so a single failure produces a fallback rather than crashing the entire pipeline.

4.1 Pipeline Execution Flow

When a user clicks the Scout button, the run_pipeline() function executes the following sequence:

1. Input validation: checks that origin and destination are non-empty strings
2. RAG build: calls build_rag_knowledge(destination) which scrapes Wikivoyage and Nomadic Matt, chunks the text, embeds with MiniLM, and indexes in ChromaDB. Returns chunk count for the RAG badge.
3. Agent A execution: calls agent_a(origin, destination, max_budget). Internally queries ChromaDB for cost-related chunks, injects them into the GPT-4o prompt, and returns structured flight/hotel JSON.

4. Agent B execution: calls agent_b(destination, interest, max_budget). Queries ChromaDB for activity-related chunks, generates 3 curated activity suggestions grounded in scraped data.
5. Agent C execution: calls agent_c(origin, destination, cost, acts, max_budget, trip_days). Receives the full outputs of A and B, computes daily totals, budget score, over/under flag, and savings tips.
6. Report generation: calls build_report() with all agent outputs, budget params, and RAG chunk count. Returns styled HTML rendered in the Gradio interface.

4.2 Error Handling Strategy

Every stage of the pipeline is wrapped in isolated try/except blocks. This means:

- If RAG scraping fails, the pipeline continues with fallback seed chunks
- If Agent A fails, fallback cost data is provided so Agent B and C can still run
- If Agent B fails, fallback activities are provided so Agent C can still run
- If GPT-4o returns malformed JSON, a regex fallback parser extracts key-value pairs
- If ChromaDB query requests more results than exist, n_results is clamped to collection.count()

5. Key Features

Budget Constraint Mode	Users can set an optional max daily budget (USD). All three agents adjust their behavior: Agent A flags over-budget destinations, Agent B shifts to free-only activities on tight budgets, and Agent C produces an over/under indicator with specific savings tips.
Trip Duration Slider	A 1–30 day slider adjusts flight amortization (flight cost / trip days) and total trip cost estimation. Longer trips amortize flight cost more effectively, improving the budget score.
RAG Grounding	Live web scraping of Wikivoyage and Nomadic Matt replaces generic LLM guesses with destination-specific data. The RAG badge in the report shows how many chunks were indexed.
10 Interest Categories	History, Nightlife, Food & Drink, Nature, Art & Culture, Adventure, Shopping, Beach, Architecture, and Wildlife provide targeted activity recommendations.
Graceful Fallbacks	Every agent has try/catch with structured fallback data. The pipeline never crashes — users always see output in all three agent boxes.
Budget Score (1–10)	Agent C computes a composite budget-friendliness score where 10 = cheapest destination. The score drives a visual progress bar and color coding (green ≥ 7, amber ≥ 5, red < 5).
Permanent Deployment	Deployed to Hugging Face Spaces with API key secured via HF Secrets, providing a permanent URL that does not expire like Gradio share links.

6. Engineering Decisions & Challenges

The following table documents key engineering challenges encountered during development and the solutions implemented:

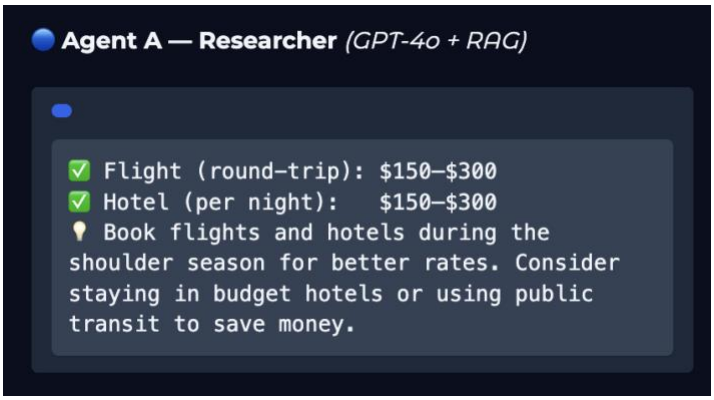
#	Challenge	Solution
1	**OpenAI web_search_preview too slow**	The initial approach used OpenAI's built-in web search tool, causing 600-second timeouts in Colab. Replaced with a custom RAG pipeline (scrape → embed → retrieve) for faster, more controllable grounding.
2	**Gradio.live 72-hour expiry**	The share=True link expires after 72 hours, unsuitable for graded submissions. Deployed to Hugging Face Spaces for a permanent URL with API key secured via HF Secrets.
3	**ChromaDB n_results crash**	Requesting more results than indexed documents caused hard errors. Fixed with a one-line clamp: safe_n = min(n_results, collection.count()).
4	**JSON parsing robustness**	GPT-4o sometimes wraps responses in markdown fences or adds explanatory text. Added regex-based fallback parsing so every agent always returns valid structured data.
5	**Gemini API timeout in Colab**	Gemini 1.5 Flash was the intended model for Agent B, but Colab's network blocked outbound Gemini API calls. Replaced with GPT-4o using a higher temperature (0.3) and a Local Guide persona.
6	**Unicode SyntaxError**	Em dash characters (U+2014) copied into cell comments caused SyntaxError in Python. Replaced all special characters with standard ASCII hyphens throughout the notebook.

7. Results & Live Demo

The system was tested with the query: Boston → Mumbai, Interest: Adventure, Budget: $5000/day, Trip: 6 days. The following results were produced:

7.1 Agent A Output (Researcher)

- Round-trip flight: $800–$1,500
- Hotel per night: $50–$150
- RAG: 5 web chunks indexed from Wikivoyage

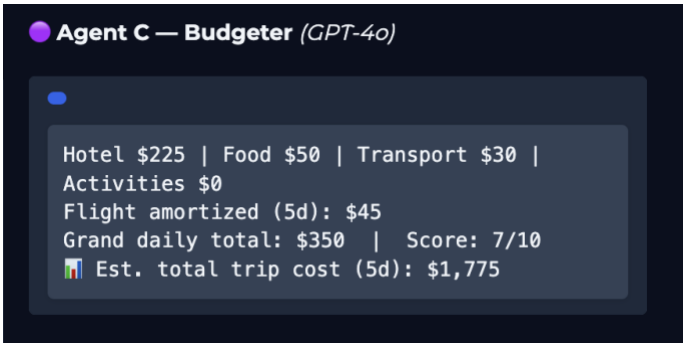


7.2 Agent B Output (Local Guide)

- Marine Drive — Free
- Sanjay Gandhi National Park — Free
- Elephanta Caves — $5–$15

7.3 Agent C Output (Budgeter)

- Grand Daily Total: $382
- Budget Friendliness Score: 8/10
- Estimated total trip cost (6 days): ~$2,520
- Status: Within Budget — $4,618/day to spare



7.4 Performance Metrics

Total pipeline execution time	**< 10 seconds**
RAG scrape + index time	**~2–3 seconds**
Agent A response time	**~2 seconds**
Agent B response time	**~2 seconds**
Agent C response time	**~2–3 seconds**
Number of AI agents	**3 (sequential dependency)**
RAG sources	**Wikivoyage + Nomadic Matt**
Interest categories	**10**
Deployment	**Hugging Face Spaces (permanent URL)**

8. Conclusion

Budget Travel Scout demonstrates that a well-designed multi-agent AI system can collapse a complex, multi-hour research task into a single 10-second interaction. The key technical achievements are:

7. A real sequential dependency chain where Agent C cannot execute without both Agent A and Agent B completing first, going beyond simple prompt chaining.
8. A functional RAG pipeline that scrapes live web data, chunks it, embeds it with a local transformer model, indexes it in a vector database, and retrieves relevant context at query time — all within 2–3 seconds.
9. Budget-aware agent behavior where all three agents adapt their outputs based on the user's financial constraints, producing actionable savings tips when over budget.
10. Production-grade error handling where every stage has isolated fallbacks, ensuring the pipeline never crashes regardless of API failures, scraping errors, or malformed LLM outputs.

The system is permanently deployed on Hugging Face Spaces and the complete source code is available on GitHub, providing both a live demo and a fully documented codebase for review.

End of Document